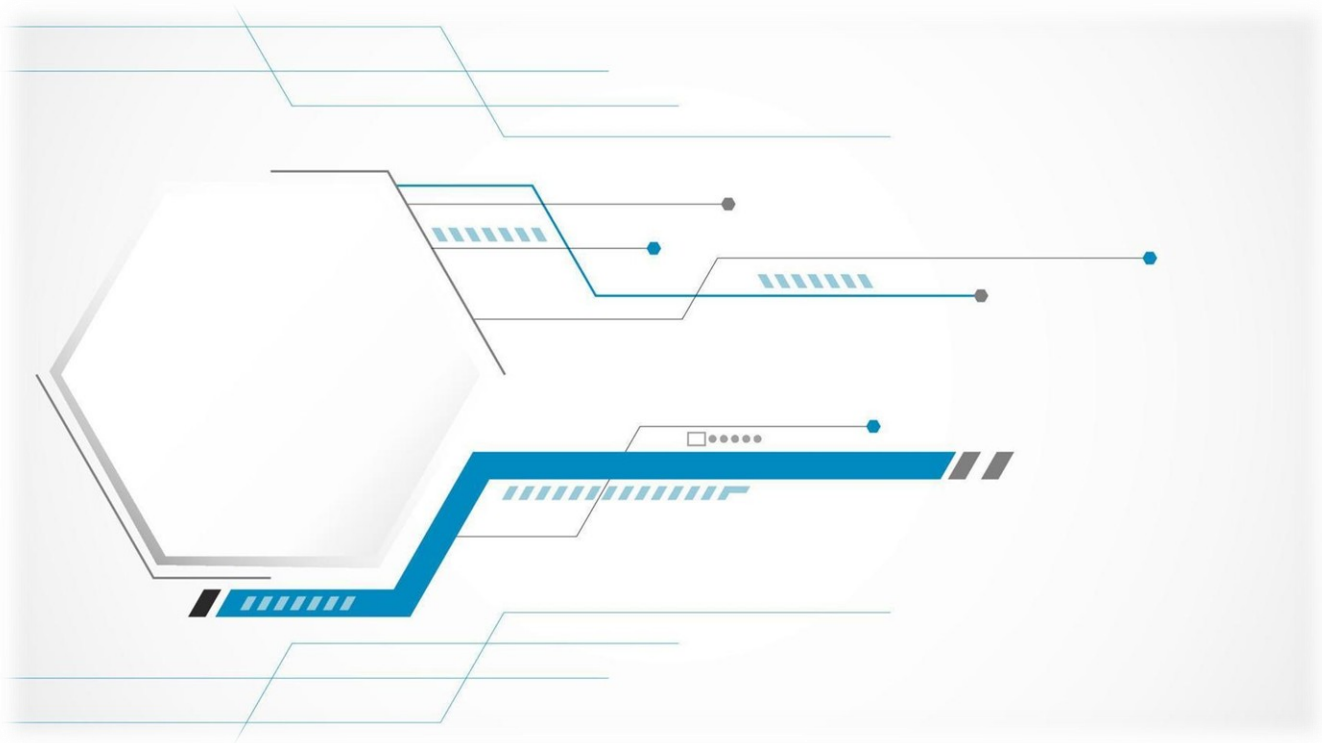


WEB TECHNOLOGY II

Complete Question Bank with Full Solutions



Q1. Define client-server architecture. Also, compare different architectures with their uses in the development of web applications. [8 Marks]

In Simple English

Client-server architecture just means one program (the client, like your browser) asks for something, and another program (the server) provides it. Depending on how many separate layers/tiers a system splits this into, we get different architecture types.

Definition

Client-server architecture is a computing model where tasks are divided between service providers (servers), which supply resources or services, and service requesters (clients), which consume them, typically communicating over a network using a defined protocol (e.g., HTTP).

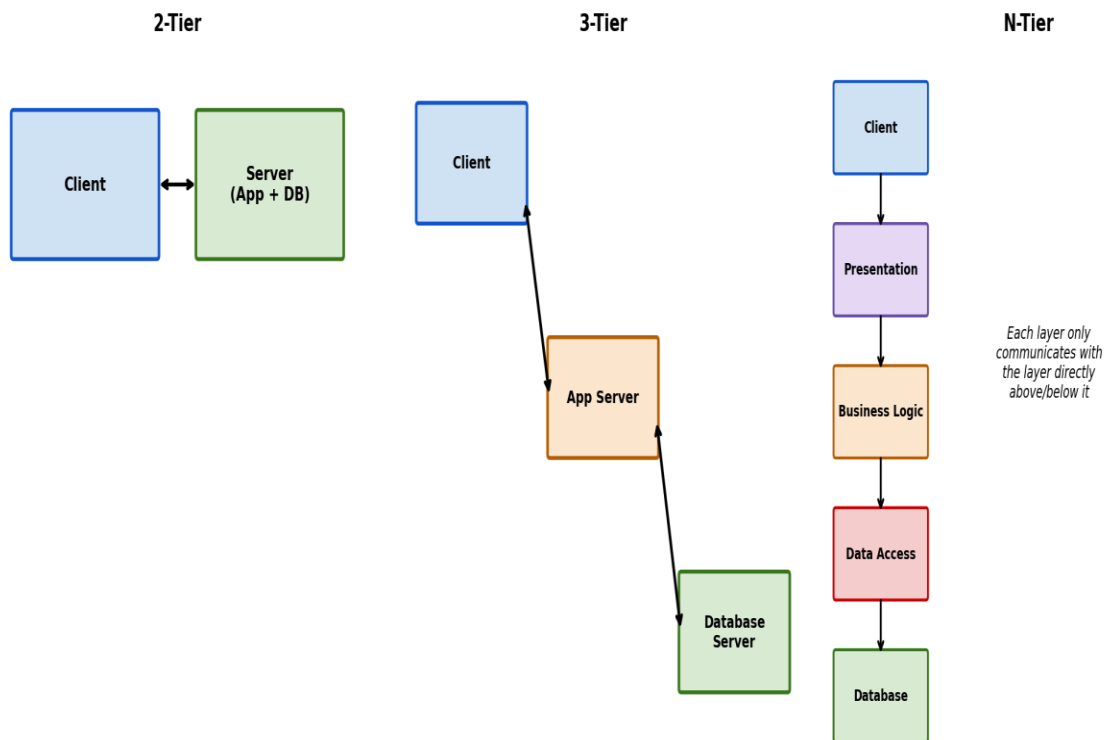


Figure: 2-Tier, 3-Tier, and N-Tier Client-Server Architectures

Comparison of Architectures

Architecture	Description	Typical Use
2-Tier	Client communicates directly with a combined application+database server	Small desktop/LAN applications with simple logic
3-Tier	Separates Presentation (client), Application/Business Logic (app server), and Data (database server) into 3 distinct layers	Most modern web applications (e.g., Flask app + separate database server)
N-Tier	Further splits the application layer into multiple specialized layers/services	Large enterprise systems, microservices-based applications

3-tier and N-tier architectures are preferred for modern web development because they separate concerns, allow each layer to scale and be updated independently, and improve security by isolating the database from direct client access.

Q2. What do you mean by Python data structure? Explain with code example of each of them. [8 Marks]

In Simple English

A data structure is just a way of organizing and storing multiple pieces of data together, so they can be accessed and manipulated efficiently. Python provides several built-in ones, each suited to different needs.

Technical Explanation

A Python data structure is a built-in or user-defined format for organizing, storing, and managing a collection of data values, enabling efficient access and manipulation. Python's core built-in data structures are List, Tuple, Dictionary, and Set.

List (ordered, mutable)

```
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")      # add an item
fruits[0] = "orange"        # modify an item (lists are mutable)
print(fruits)               # ['orange', 'banana', 'cherry', 'mango']
```

Tuple (ordered, immutable)

```
coordinates = (10, 20)
x, y = coordinates          # unpacking
print(x, y)                 # 10 20
# coordinates[0] = 5        # Error! tuples cannot be modified
```

Dictionary (key-value pairs)

```
student = {"name": "Ram", "age": 21, "faculty": "BIT"}
student["age"] = 22          # update value
print(student["name"])       # Ram
```

Set (unordered, unique elements)

```
colors = {"red", "green", "blue"}
colors.add("green")          # duplicate ignored
colors.add("yellow")
print(colors)                # {'red', 'green', 'blue', 'yellow'}
```

Q3. Define inheritance in Python. Create a base class Account and a derived class ATM as specified, with an object demonstrating its methods. [8 Marks]

In Simple English

Inheritance lets a new class reuse and extend an already-existing class, instead of writing everything from scratch. Here, ATM will reuse everything Account already has, and add its own extra features.

Definition

Inheritance is an object-oriented programming mechanism where a new class (child/derived class) acquires the attributes and methods of an existing class (parent/base class), enabling code reuse and the modeling of 'is-a' relationships.

Code Example

```
class Account:
    def __init__(self, name, account_no):
        self.name = name
        self.account_no = account_no

class ATM(Account):
    def __init__(self, name, account_no, balance):
        super().__init__(name, account_no)  # reuse parent's constructor
        self.balance = balance

    def display_account_info(self):
        print(f"Name: {self.name}, A/C No: {self.account_no}, Balance: Rs.{self.balance}")

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited Rs.{amount}. New Balance: Rs.{self.balance}")

    def withdrawal(self, amount):
        if amount > self.balance:
            print("Insufficient balance!")
        else:
            self.balance -= amount
            print(f"Withdrew Rs.{amount}. New Balance: Rs.{self.balance}")

# Creating an object of the child class
atm1 = ATM("Ram Sharma", "AC1001", 5000)
atm1.display_account_info()
atm1.deposit(2000)
atm1.withdrawal(1500)
atm1.display_account_info()
```

Here, ATM inherits name and account_no from Account, and super().__init__() calls the parent class's constructor to initialize those inherited attributes, while ATM itself adds balance and its own three methods.

Q4. What is the use of Templates and Template Inheritance? Explain different Jinja2 syntax for Flask applications in detail with a code example. [8 Marks]

In Simple English

Templates let you build HTML pages that show dynamic data (like a username) without writing raw HTML by hand every time. Template inheritance lets multiple pages share the same overall layout (header, footer, navbar), so you only need to define that layout once.

Templates

A template is an HTML file containing placeholders and simple logic (written in a templating language like Jinja2) that gets filled in with actual data at runtime, allowing dynamic content generation while keeping HTML and Python logic separated.

Template Inheritance

Template inheritance allows a 'base' template defining the common page structure (e.g., header, navigation, footer) to be extended by 'child' templates, which only need to fill in the parts that differ — avoiding repetition of common layout code across every page.

Common Jinja2 Syntax

Syntax	Purpose
{{ variable }}	Outputs the value of a variable
{% if condition %} ... {% endif %}	Conditional logic
{% for item in list %} ... {% endfor %}	Looping over a collection
{% block name %} ... {% endblock %}	Defines a replaceable section for child templates
{% extends "base.html" %}	Inherits from a base template
{{ url_for('route_name') }}	Generates a URL for a given Flask route

Code Example — base.html

```
<!DOCTYPE html>
<html>
<head><title>{% block title %}My Site{% endblock %}</title></head>
<body>
  <nav>Welcome to My Website</nav>
  {% block content %}{% endblock %}
  <footer>&copy; 2026</footer>
</body>
</html>
```

Code Example — home.html (child template)

```
{% extends "base.html" %}
{% block title %}Home Page{% endblock %}
{% block content %}
    <h1>Hello, {{ username }}!</h1>
    {% if logged_in %}
        <p>You are logged in.</p>
    {% else %}
        <p>Please log in.</p>
    {% endif %}
    <ul>
    {% for item in items %}
        <li>{{ item }}</li>
    {% endfor %}
    </ul>
{% endblock %}
```

Q5. Define Relational and NoSQL databases. Explain how SQLAlchemy reduces development time and produces a secure system. [8 Marks]

In Simple English

Relational databases store data in strict, related tables (like Excel sheets that reference each other). NoSQL databases store data more flexibly, without needing a rigid table structure. SQLAlchemy is a Python tool that lets you work with databases using plain Python code instead of writing raw, error-prone SQL by hand.

Relational Databases

A relational database organizes data into structured tables with predefined schemas, rows, and columns, using relationships (via foreign keys) to connect related data, and is queried using SQL. Examples: MySQL, PostgreSQL, SQLite.

NoSQL Databases

A NoSQL ('Not Only SQL') database stores data in flexible, schema-less formats — such as documents (MongoDB), key-value pairs (Redis), or graphs (Neo4j) — well-suited to unstructured or rapidly-changing data and horizontal scalability.

How SQLAlchemy Reduces Development Time and Improves Security

- **Object-Relational Mapping (ORM):** Lets developers work with Python classes/objects instead of writing raw SQL, speeding up development and reducing boilerplate code.
- **Database Portability:** The same Python code can work across different database engines (SQLite, MySQL, PostgreSQL) with minimal changes.
- **Automatic Parameterized Queries:** SQLAlchemy automatically escapes and parameterizes query values, protecting against SQL Injection attacks — a major security improvement over manually concatenated SQL strings.
- **Migration Support:** Tools like Alembic (built for SQLAlchemy) manage schema changes safely and consistently over time.
- **Reduced Human Error:** Since developers work with validated Python objects rather than raw strings, many common SQL syntax and typing errors are eliminated.

Q6. Design a signup page having 4-5 fields and encrypt the password, saving all the information to a database using SQLAlchemy. Also, write code to display all the users. [8 Marks]

In Simple English

This builds a working registration form: the user fills in their details, the password gets scrambled (hashed) before saving so it's never stored in plain readable text, and everything is saved into the database. A second page then lists everyone who signed up.

Code Example — Models and Setup

```
from flask import Flask, render_template, request, redirect, url_for
from flask_sqlalchemy import SQLAlchemy
from werkzeug.security import generate_password_hash

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    full_name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False)
    phone = db.Column(db.String(20))
    address = db.Column(db.String(200))
    password_hash = db.Column(db.String(200), nullable=False)

with app.app_context():
    db.create_all()
```

Code Example — Signup Route

```
@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        new_user = User(
            full_name=request.form['full_name'],
            email=request.form['email'],
            phone=request.form['phone'],
            address=request.form['address'],
            password_hash=generate_password_hash(request.form['password'])
        )
        db.session.add(new_user)
        db.session.commit()
        return redirect(url_for('users'))
    return render_template('signup.html')
```

Code Example — Display All Users

```
@app.route('/users')
def users():
    all_users = User.query.all()
    return render_template('users.html', users=all_users)

if __name__ == '__main__':
    app.run(debug=True)
```

The password is never stored in readable form — `generate_password_hash()` converts it into an irreversible hash before saving, so even if the database is compromised, the actual passwords remain protected.

Q7. Define CMS. Also, differentiate the popular CMS in detail. [8 Marks]

In Simple English

A CMS is software that lets you build and manage a website's content (text, images, pages) through a simple visual interface, without needing to write code for every page.

Definition

A Content Management System (CMS) is a software application that enables users to create, manage, edit, and publish digital content on a website through a user-friendly interface, without requiring specialized technical/coding knowledge.

Popular CMS Compared

CMS	Built With	Best Suited For	Key Characteristic
WordPress	PHP	Blogs, small-to-medium business sites	Largest ecosystem of themes/plugins; very beginner-friendly
Joomla	PHP	Medium-complexity sites, community platforms	More built-in features than WordPress but steeper learning curve
Drupal	PHP	Large, complex, highly-customized enterprise sites	Highly flexible/scalable but requires more technical expertise
Magento	PHP	E-commerce websites	Purpose-built for online stores with advanced catalog/inventory features
Shopify	Ruby on Rails (SaaS)	E-commerce (hosted solution)	Fully managed, hosted platform — no server management needed

Q8. Write short notes on: Session and Cookies. [8 Marks]

In Simple English

Since the web's HTTP protocol forgets everything between requests, cookies and sessions are the two main tricks used to make a website 'remember' who you are as you move from page to page.

Cookies (Client-Side) vs Sessions (Server-Side)

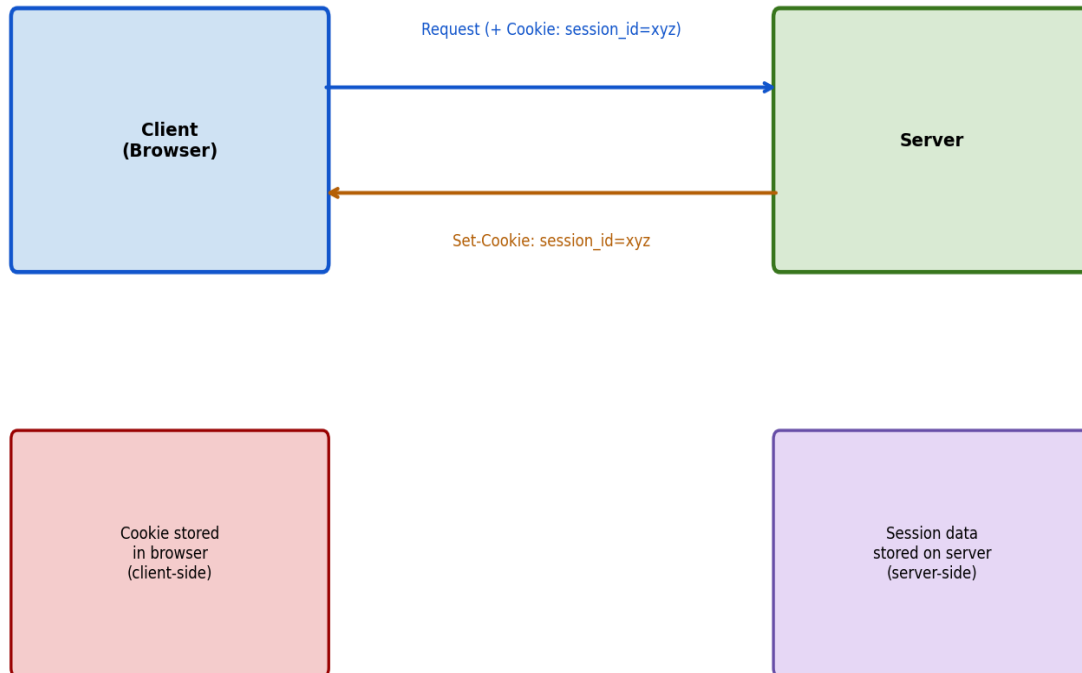


Figure: Cookies (Client-Side) vs Sessions (Server-Side)

Cookies

A cookie is a small piece of data that a web server sends to the client's browser, which the browser stores and automatically sends back with every subsequent request to that same server — commonly used to remember user preferences, login tokens, or track activity.

Sessions

A session is a server-side mechanism for storing data about a specific user's interaction across multiple requests, identified by a unique session ID (usually stored in a cookie on the client). Unlike cookie data, the actual session data lives securely on the server, not in the browser.

Key Difference

Cookies are stored on the client and are visible/editable by the user; sessions are stored on the server and are only referenced by a cookie holding the session ID — making sessions generally more secure for storing sensitive data like login state.

Q9. Write short notes on: Sending Email in Flask. [8 Marks]

In Simple English

Flask applications commonly need to send emails — for account verification, password resets, or notifications. The Flask-Mail extension makes this straightforward.

Technical Explanation

Flask supports sending emails through extensions such as Flask-Mail, which wraps Python's built-in `smtpplib` to integrate email-sending directly into a Flask application's configuration and route handlers.

```
from flask_mail import Mail, Message

app.config['MAIL_SERVER'] = 'smtp.gmail.com'
app.config['MAIL_PORT'] = 587
app.config['MAIL_USE_TLS'] = True
app.config['MAIL_USERNAME'] = 'youremail@gmail.com'
app.config['MAIL_PASSWORD'] = 'your-app-password'
mail = Mail(app)

@app.route('/send-mail')
def send_mail():
    msg = Message('Welcome!', sender='youremail@gmail.com', recipients=['user@example.com'])
    msg.body = 'Thank you for signing up.'
    mail.send(msg)
    return 'Email sent!'
```

Q10. Write short notes on: Gunicorn. [8 Marks]

In Simple English

Flask's own built-in server is only meant for testing during development — Gunicorn is a much sturdier server used to actually run Flask (and other Python) applications in production, handling many real users at once.

Technical Explanation

Gunicorn ('Green Unicorn') is a WSGI (Web Server Gateway Interface) HTTP server for Python web applications, commonly used to serve Flask and Django applications in production. It runs multiple worker processes to handle concurrent requests efficiently, and is typically deployed behind a reverse proxy like Nginx.

```
gunicorn --workers 4 --bind 0.0.0.0:8000 app:app
```

This command starts the Flask application ('app' object inside `app.py`) using 4 worker processes, bound to port 8000 — offering far better performance, stability, and concurrency handling than Flask's built-in development server.

Q1. What is Server? Compare monolithic and microservices architectures in terms of scalability, deployment, and maintenance. [8 Marks]

In Simple English

A server is a computer/program that provides services or resources to other programs (clients). Monolithic means building an app as one single, big connected block; microservices means splitting it into many small, independent pieces that each do one job.

What is a Server?

A server is a computer program or hardware device that provides functionality, resources, or services to other programs or devices (clients) over a network, typically listening for and responding to client requests.

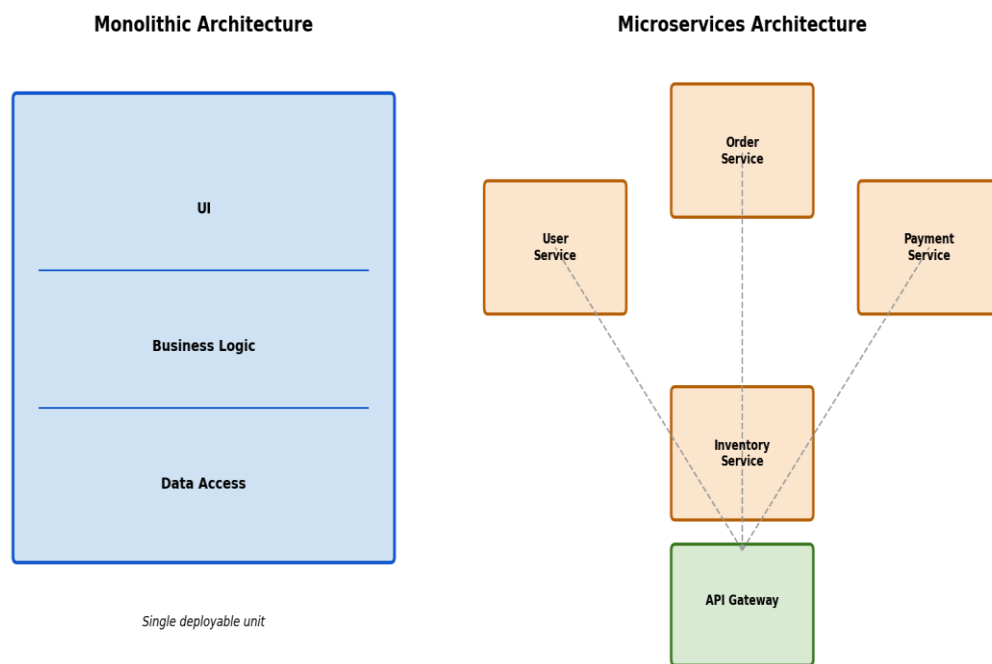


Figure: Monolithic vs Microservices Architecture

Monolithic vs Microservices

Aspect	Monolithic Architecture	Microservices Architecture
Structure	Single, unified codebase and deployable unit	Application split into many small, independent services
Scalability	Must scale the entire application together, even if only one part needs it	Each service can be scaled independently based on its own demand
Deployment	One deployment for the whole application — simpler but riskier (any change requires redeploying everything)	Independent deployment per service — more complex tooling but lower risk per change
Maintenance	Easier initially, but grows harder as the codebase grows large and tangled	Easier to maintain individual services long-term, but requires managing inter-service communication and coordination

Q2. Write a Python program to demonstrate the use of lists, tuples, and dictionaries with at least three operations on each. [8 Marks]

```
# --- List operations ---
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")          # 1. Add an item
fruits.remove("banana")        # 2. Remove an item
print(fruits[1])               # 3. Access by index

# --- Tuple operations ---
dimensions = (1920, 1080, "HD")
print(dimensions[0])           # 1. Access by index
print(len(dimensions))         # 2. Get length
a, b, c_ = dimensions          # 3. Unpacking

# --- Dictionary operations ---
student = {"name": "Sita", "age": 20}
student["faculty"] = "BIT"     # 1. Add a new key-value pair
student["age"] = 21            # 2. Update a value
print(student.keys())          # 3. Get all keys
```

Q3. What is a function in Python? Write a Python program using functions to calculate the electricity bill as per the given slab conditions. [8 Marks]

In Simple English

A function is a reusable, named block of code that performs a specific task — you 'call' it whenever you need that task done, instead of rewriting the same logic repeatedly.

Definition

A function in Python is a block of organized, reusable code defined using the 'def' keyword, that performs a specific task and can optionally accept input parameters and return a value.

Code Example — Electricity Bill Calculator

```
def calculate_bill(units):
    if units <= 100:
        bill = units * 5
    elif units <= 200:
        bill = (100 * 5) + (units - 100) * 7
    else:
        bill = (100 * 5) + (100 * 7) + (units - 200) * 10
    return bill

units_consumed = int(input("Enter units consumed: "))
total = calculate_bill(units_consumed)
print(f"Total Electricity Bill: Rs.{total}")
```

Example: for 250 units, bill = (100×5) + (100×7) + (50×10) = 500 + 700 + 500 = Rs. 1700.

Alternative — CSV File Writing Program

(This paper offered a choice between the electricity bill program above or the following CSV program — both are provided for completeness.)

```
import csv

with open('students.csv', 'w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(['Name', 'Address', 'Mobile', 'Age'])
    for i in range(10):
        name = input("Enter name: ")
        address = input("Enter address: ")
        mobile = input("Enter mobile number: ")
        age = input("Enter age: ")
        writer.writerow([name, address, mobile, age])

print("Student details saved to students.csv")
```

**Q4. What is a framework, and why is it important for developing a secure application?
Explain why Flask is called a micro framework. [8 Marks]**

In Simple English

A framework is a pre-built foundation/toolkit that handles a lot of the repetitive, error-prone plumbing work of building an application, so developers can focus on the app's actual logic instead of reinventing basic infrastructure (and making security mistakes in the process).

Definition

A web framework is a software toolkit that provides a structured foundation — reusable libraries, tools, and conventions — for building web applications, handling common tasks such as routing, request/response handling, and templating.

Importance for Secure Application Development

- Frameworks often include built-in protections against common vulnerabilities (e.g., automatic escaping in templates prevents XSS; ORM layers prevent SQL Injection)
- Provide tested, community-vetted implementations of security-sensitive features (session management, password hashing) rather than requiring developers to write these from scratch
- Enforce consistent structure and coding conventions, reducing the chance of ad-hoc, insecure code
- Frequently updated with security patches by an active maintainer community

Why Flask Is Called a Micro Framework

Flask is called a 'micro' framework because it provides only the essential core components needed to build a web application — routing, request handling, and templating (via Jinja2) — without bundling in additional features like built-in ORM, form validation, or authentication by default. Instead, Flask is intentionally minimal and extensible, letting developers add exactly the extensions they need (e.g., Flask-SQLAlchemy, Flask-Login, Flask-WTF) — unlike 'batteries-included' frameworks such as Django.

Q5. What is a database? Why is SQLAlchemy used with Flask? Compare ORM and Core approaches in SQLAlchemy. [8 Marks]

In Simple English

A database is an organized place to store data permanently. SQLAlchemy is used with Flask to let developers interact with that database using Python code instead of raw SQL, and it offers two styles of doing so: a high-level ORM style and a lower-level Core style.

Definition

A database is an organized collection of structured data, stored electronically and managed by a database management system (DBMS), enabling efficient storage, retrieval, and manipulation of data.

Why SQLAlchemy Is Used with Flask

- Provides an abstraction layer over raw SQL, letting developers write database code in Python
- Protects against SQL Injection through automatic query parameterization
- Supports multiple database engines with minimal code changes (portability)
- Integrates cleanly with Flask via the Flask-SQLAlchemy extension, simplifying configuration and session management

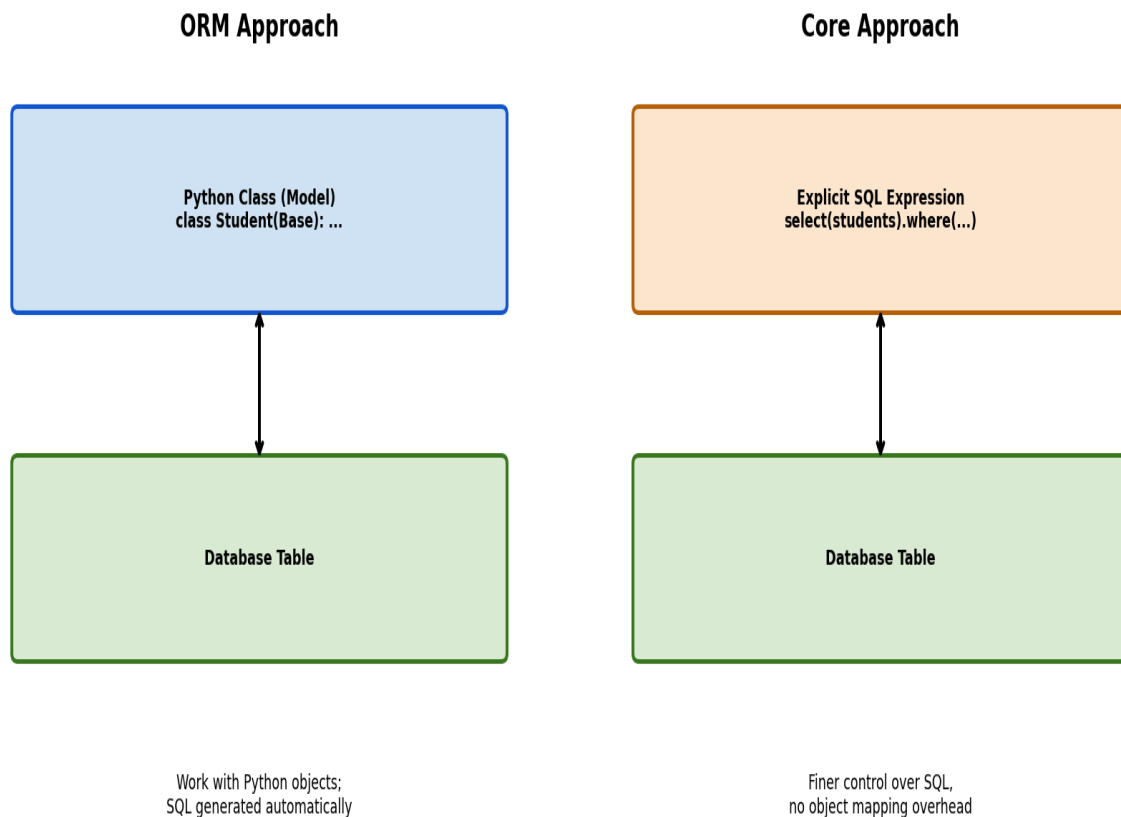


Figure: SQLAlchemy ORM vs Core Approach

ORM vs Core

Aspect	ORM Approach	Core Approach
Style	Work with Python classes/objects representing tables	Work directly with SQL Expression Language constructs
Abstraction Level	Higher-level, more Pythonic	Lower-level, closer to raw SQL
Control	Less fine-grained control over generated SQL	More precise control over exact SQL generated
Best For	Typical CRUD-heavy application development (rapid development)	Performance-critical or complex queries needing fine control

Q6. Differentiate between sessions and cookies. Demonstrate sending email in Flask with a suitable example. [8 Marks]

Sessions vs Cookies

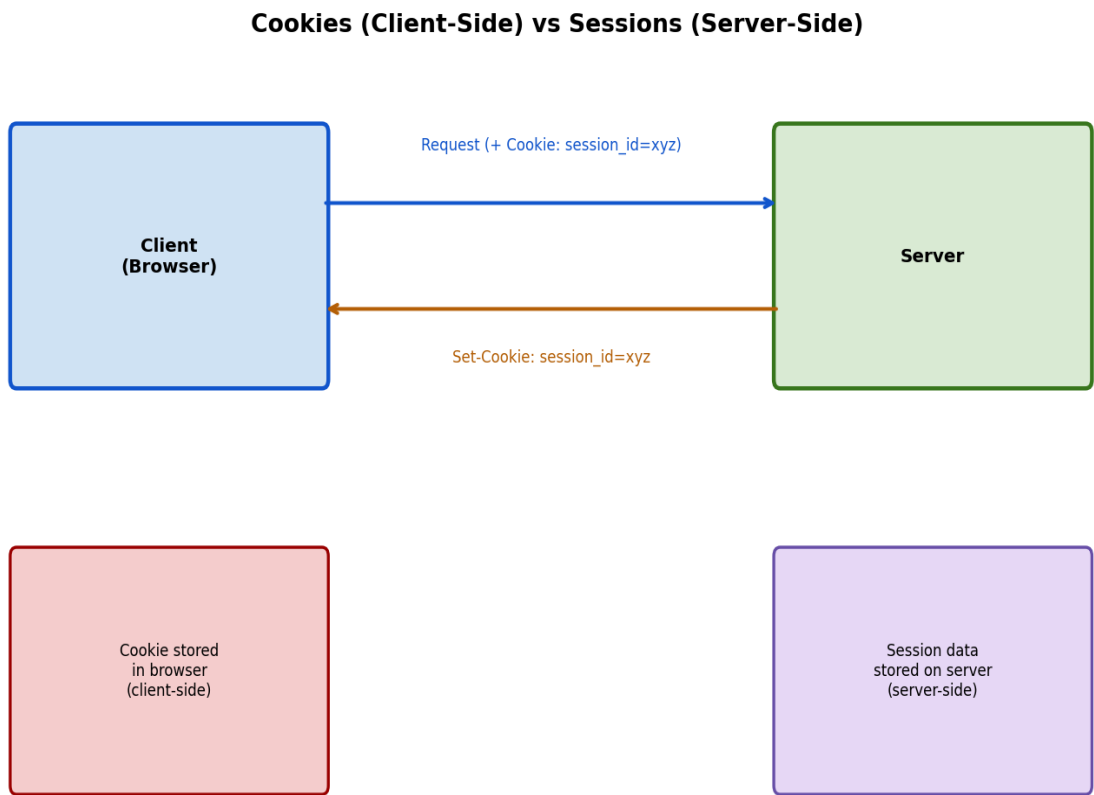


Figure: Cookies (Client-Side) vs Sessions (Server-Side)

Aspect	Cookies	Sessions
Storage Location	Client-side (browser)	Server-side
Security	Less secure — visible/editable on client	More secure — actual data hidden from client
Size Limit	Limited (~4KB per cookie)	Limited only by server storage/memory
Lifespan	Can persist across browser sessions if given an expiry	Typically expires when the browser closes or after a timeout

Sending Email in Flask — Example

```
from flask_mail import Mail, Message

app.config['MAIL_SERVER'] = 'smtp.gmail.com'
app.config['MAIL_PORT'] = 587
app.config['MAIL_USE_TLS'] = True
app.config['MAIL_USERNAME'] = 'youremail@gmail.com'
app.config['MAIL_PASSWORD'] = 'your-app-password'
mail = Mail(app)

@app.route('/notify')
def notify():
    msg = Message('Order Confirmed', sender='youremail@gmail.com',
recipients=['customer@example.com'])
    msg.body = 'Your order has been confirmed. Thank you!'
    mail.send(msg)
    return 'Notification email sent!'
```

Q7. What is a Content Management System (CMS)? Explain its features and advantages. Also, list some popular CMS's. [8 Marks]

Definition

A CMS is a software application that enables users to create, manage, edit, and publish digital content on a website through a user-friendly interface, without requiring specialized technical/coding knowledge.

Features

- WYSIWYG (What You See Is What You Get) content editor
- User and role management (different permission levels)
- Template/theme system for consistent design
- Plugin/extension ecosystem for adding functionality
- Media management for images, videos, and documents
- SEO tools for optimizing content visibility

Advantages

- Non-technical users can create and update content without programmer involvement
- Faster website development using pre-built themes and plugins
- Centralized content management across large sites
- Strong community support and extensive documentation for popular CMS platforms

Popular CMS

WordPress, Joomla, Drupal, Magento, and Shopify — see the comparison table in the Year 2026 section for detailed differences.

Q8. Write a Python Flask application that connects to a database using SQLAlchemy, fetching all records from a students table and displaying them in an HTML table. [8 Marks]

```
from flask import Flask, render_template
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///students.db'
db = SQLAlchemy(app)

class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100))
    email = db.Column(db.String(100))
    gpa = db.Column(db.Float)

@app.route('/students')
def show_students():
    students = Student.query.all()
    return render_template('students.html', students=students)

if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(debug=True)
```

students.html

```
<table border="1">
  <tr><th>ID</th><th>Name</th><th>Email</th><th>GPA</th></tr>
  {% for s in students %}
  <tr><td>{{ s.id }}</td><td>{{ s.name }}</td><td>{{ s.email }}</td><td>{{ s.gpa }}</td></tr>
  {% endfor %}
</table>
```

Q9. What is SOAP protocol? Describe the main HTTP methods used in RESTful APIs and their purposes. [8 Marks]

In Simple English

SOAP is a strict, XML-based standard for exchanging structured information between systems. REST APIs, in contrast, mainly use standard HTTP methods to represent different actions on data.

SOAP Protocol

SOAP (Simple Object Access Protocol) is a strict, XML-based messaging protocol used for exchanging structured information between applications over a network, typically over HTTP or SMTP, with a well-defined message structure (Envelope, Header, Body).

SOAP vs REST		
	SOAP	REST
Protocol	Strict protocol (XML-based)	Architectural style (any format, usually JSON)
Transport	Can use HTTP, SMTP, others	Typically HTTP only
Message Format	XML (Envelope, Header, Body)	JSON, XML, plain text
State	Can be stateful or stateless	Stateless by design
Performance	Heavier, more verbose	Lighter, faster

Figure: SOAP vs REST Comparison

Main HTTP Methods Used in RESTful APIs

Method	Purpose
GET	Retrieve a resource (read-only, no side effects)
POST	Create a new resource
PUT	Update/replace an existing resource entirely
PATCH	Partially update an existing resource
DELETE	Remove a resource

Q10. Differentiate between development and production environments in web development. Why is it not recommended to run development servers in production? [8 Marks]

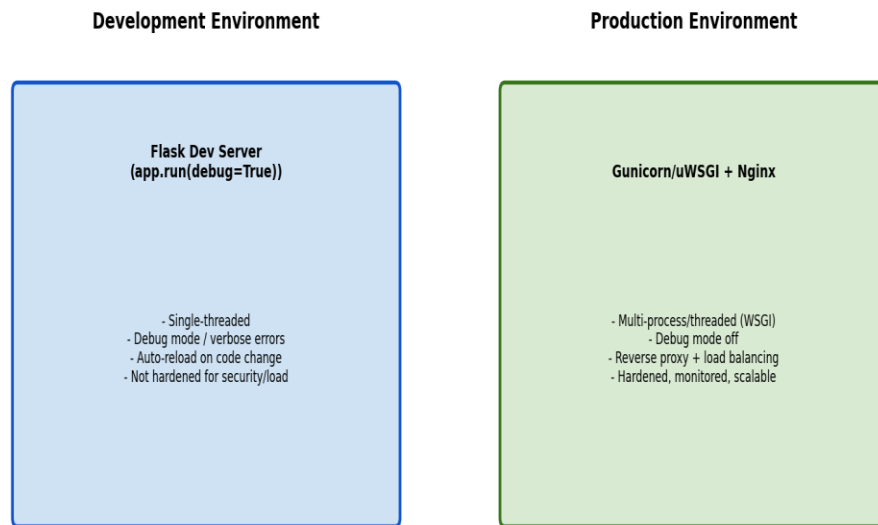


Figure: Development vs Production Environment

Aspect	Development Environment	Production Environment
Purpose	Writing, testing, and debugging code	Serving real users with the live application
Server	Flask's built-in dev server (single-threaded)	WSGI server (Gunicorn/uWSGI) behind a reverse proxy (Nginx)
Debug Mode	Enabled — shows detailed error tracebacks	Disabled — errors are logged, not exposed to users
Performance/Concurrency	Not optimized for multiple simultaneous users	Optimized for handling many concurrent requests reliably

Why Not Run Development Servers in Production

- The development server is single-threaded and cannot efficiently handle multiple concurrent users, causing poor performance/availability under real traffic
- Debug mode exposes detailed stack traces and even allows arbitrary code execution through the interactive debugger — a severe security risk if exposed publicly
- It lacks production-grade features like proper process management, automatic restarts on crashes, and load balancing
- It is explicitly documented by Flask's own maintainers as unsuitable for production use

Q11. What do you mean by threat, vulnerabilities, and attacks? Define injection and how can we be safe from it? [8 Marks]

Definitions

Threat: A potential danger that could exploit a vulnerability to cause harm to a system (e.g., a hacker attempting unauthorized access).

Vulnerability: A weakness or flaw in a system's design, implementation, or configuration that could be exploited (e.g., unsanitized user input in a form field).

Attack: An actual, deliberate attempt to exploit a vulnerability to cause harm or gain unauthorized access (e.g., an actual SQL Injection attempt submitted through a login form).

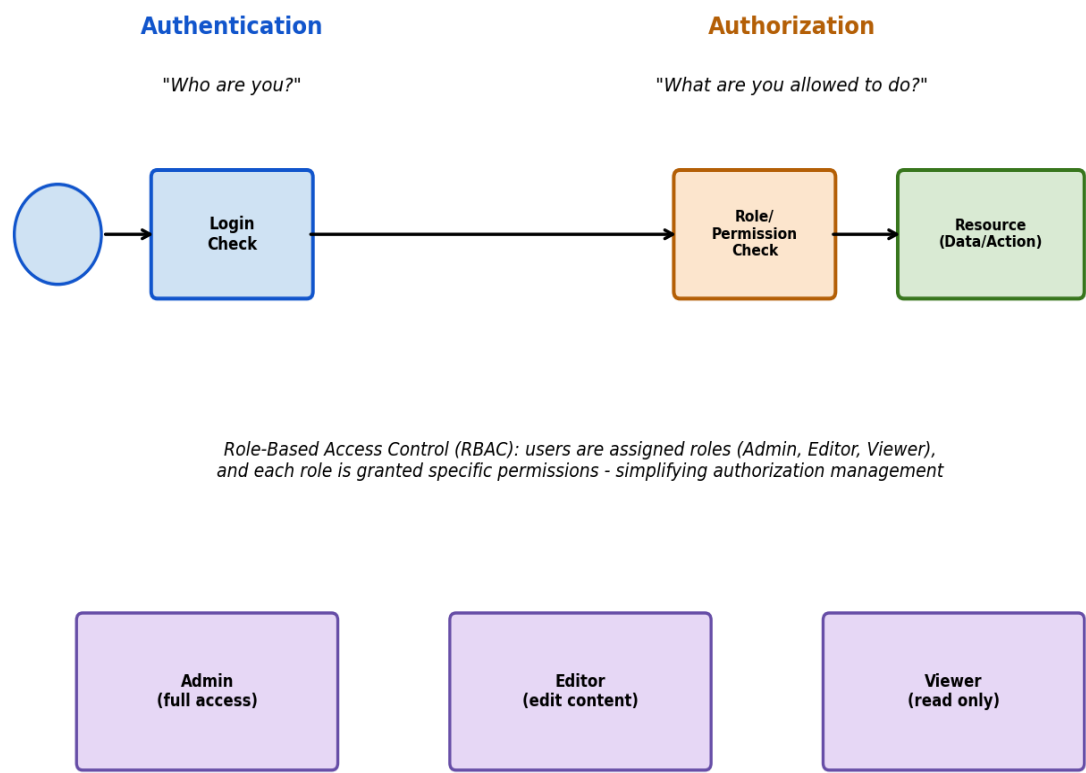
Injection

Injection is a class of vulnerability where an attacker sends malicious, untrusted input that gets interpreted as part of a command or query (e.g., SQL, shell commands) rather than as plain data, potentially allowing unauthorized data access, modification, or system compromise. SQL Injection is the most common example.

Staying Safe from Injection

- Use parameterized queries / prepared statements (or an ORM like SQLAlchemy) instead of directly concatenating user input into queries
 - Validate and sanitize all user input on both client and server side
 - Apply the principle of least privilege to database accounts used by the application
 - Use Web Application Firewalls (WAFs) as an additional layer of defense
-

Q12. What is Web Application security? Explain the necessity of Role Based Access Control. Also, differentiate between Authentication and Authorization. [8 Marks]



Role-Based Access Control (RBAC): users are assigned roles (Admin, Editor, Viewer), and each role is granted specific permissions - simplifying authorization management

Figure: Authentication vs Authorization and Role-Based Access Control

Web Application Security

Web application security refers to the practices, technologies, and processes used to protect web applications from threats and vulnerabilities that could compromise their confidentiality, integrity, or availability — including protection against injection, XSS, broken authentication, and other common attack vectors.

Necessity of Role-Based Access Control (RBAC)

RBAC restricts system access based on a user's assigned role (e.g., Admin, Editor, Viewer), rather than managing permissions individually for every user. It is necessary because it: simplifies permission management at scale, reduces the risk of privilege misconfiguration, enforces the principle of least privilege, and makes auditing access far easier.

Authentication vs Authorization

Aspect	Authentication	Authorization
Question	Who are you?	What are you allowed to do?
Occurs	First — verifies identity	After authentication — checks permissions
Example	Logging in with username/password	A logged-in 'Viewer' being blocked from deleting a record

Q13. Write short notes on: SSL/TLS. [8 Marks]

SSL/TLS Handshake (Simplified)

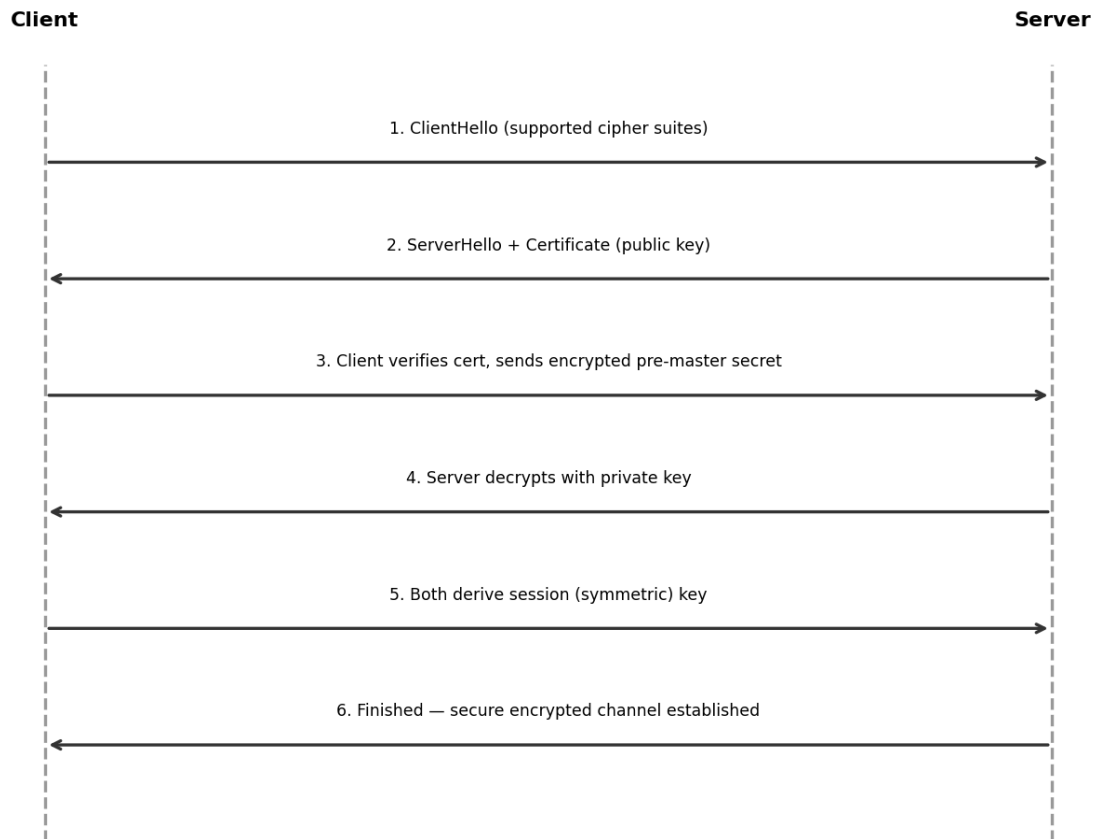


Figure: SSL/TLS Handshake (Simplified)

SSL (Secure Sockets Layer) and its modern successor TLS (Transport Layer Security) are cryptographic protocols that provide secure, encrypted communication between a client and a server over a network. During the handshake, the client and server agree on encryption algorithms, the server presents a digital certificate to prove its identity, and both sides derive a shared symmetric session key used to encrypt all subsequent communication. This underpins HTTPS, protecting data (such as passwords and payment details) from eavesdropping and tampering during transmission.

Q14. Write short notes on: MVC Architecture. *[8 Marks]*

MVC (Model-View-Controller) Architecture

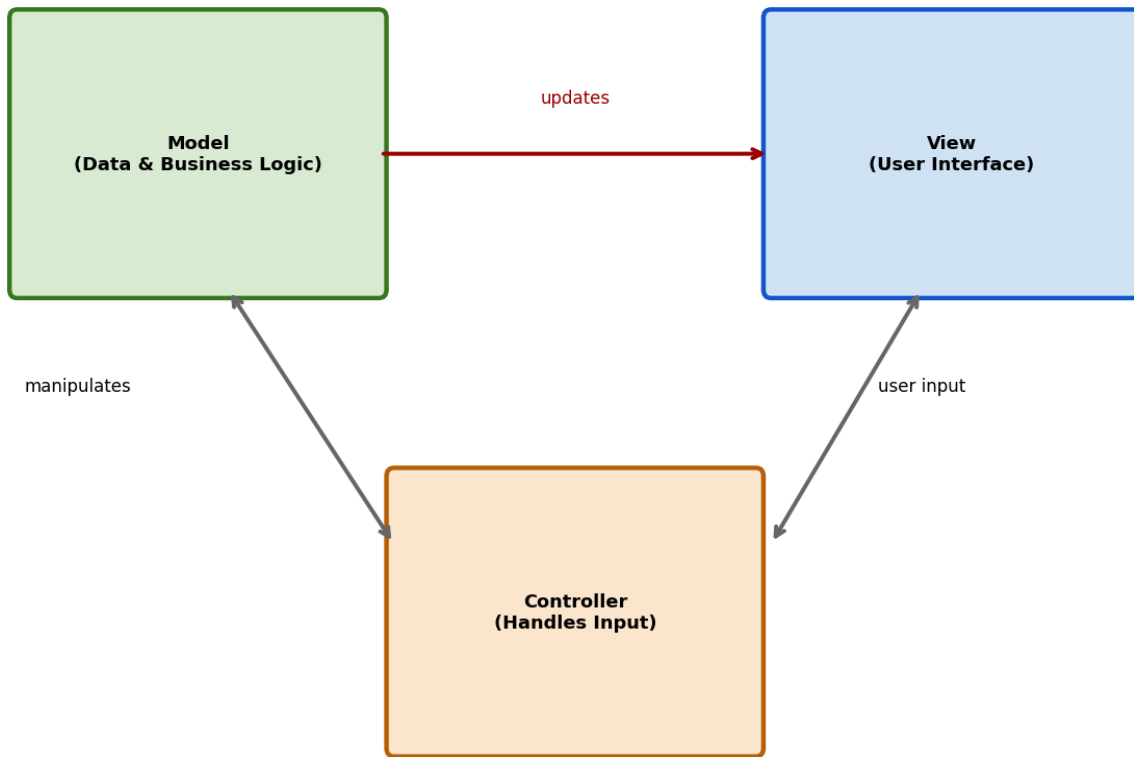


Figure: MVC (Model-View-Controller) Architecture

MVC (Model-View-Controller) is a software design pattern that separates an application into three interconnected components: the Model (manages data and business logic), the View (presents the user interface), and the Controller (handles user input and coordinates between the Model and View). This separation of concerns makes applications easier to maintain, test, and extend, since changes to one component (e.g., the UI) have minimal impact on the others. Many popular web frameworks (Django, Ruby on Rails, Laravel, ASP.NET MVC) are built around this pattern.

Q15. Write short notes on: CSV and JSON. [8 Marks]

CSV (Comma-Separated Values): A simple, plain-text file format where data is organized in rows and columns, with values separated by commas. It is widely used for exchanging tabular data (e.g., spreadsheets, database exports) due to its simplicity and broad compatibility.

JSON (JavaScript Object Notation): A lightweight, text-based data-interchange format that represents data as key-value pairs and arrays, similar to Python dictionaries/lists. It is the most common format for exchanging data in modern web APIs due to its readability and native support in JavaScript.

```
# CSV example
name,age,faculty
Ram,21,BIT
```

```
# JSON example
{ "name": "Ram", "age": 21, "faculty": "BIT" }
```

Q16. Explain the role of cookies and sessions in maintaining state between a client and a server. [8 Marks]

In Simple English

HTTP is naturally 'forgetful' — by default, a server treats every single request as if it's coming from a brand-new stranger, with no memory of any previous request. Cookies and sessions are the two tools that fix this, letting a website remember who you are and what you've been doing as you move from page to page.

The Problem: HTTP Is Stateless

HTTP is a stateless protocol — each request-response cycle is independent, and the server retains no memory of prior interactions with that same client by default. This creates a practical problem: without some mechanism to track identity across requests, a website couldn't keep you logged in, remember your shopping cart, or track your progress through a multi-step form.

Cookies (Client-Side) vs Sessions (Server-Side)

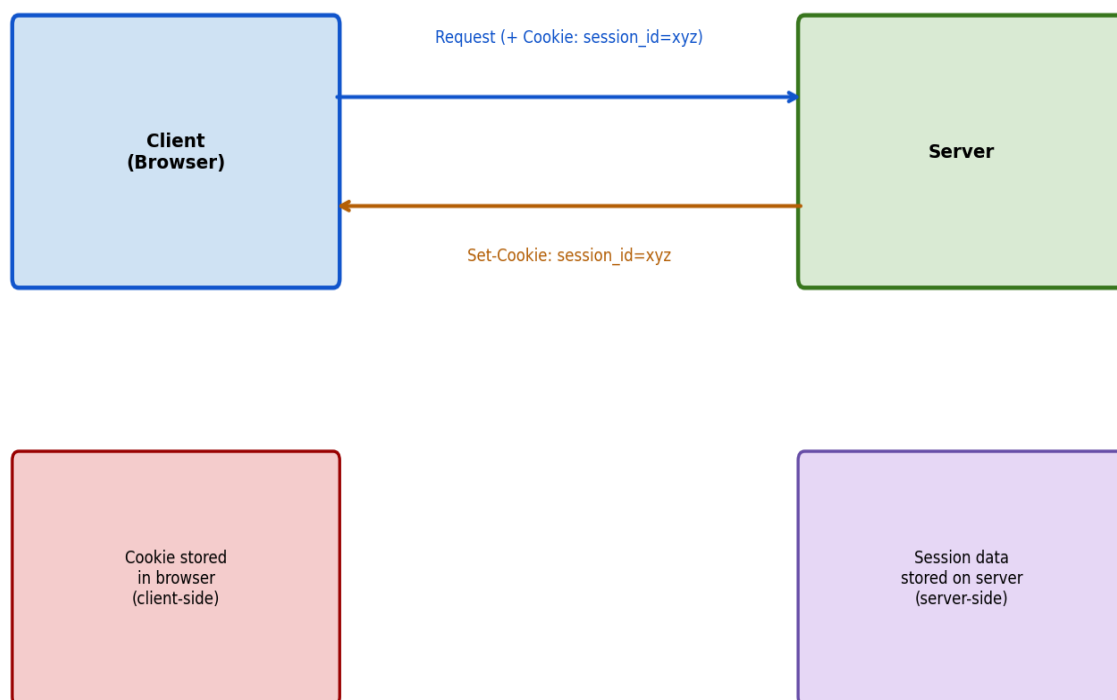


Figure: Cookies (Client-Side) vs Sessions (Server-Side)

Role of Cookies in Maintaining State

A cookie is a small piece of data the server sends to the browser (via a Set-Cookie header), which the browser then automatically attaches to every subsequent request sent to that same domain. This lets the server recognize returning requests as belonging to the same client, without the client needing to do anything extra.

Role of Sessions in Maintaining State

A session extends this idea by storing the actual state data (e.g., 'user is logged in as Ram', cart contents) securely on the server, rather than in the browser. The server generates a unique session ID for each client and sends only that ID to the browser as a cookie. On each subsequent request, the browser sends back the session ID cookie, and the server uses it to look up the corresponding session data — reconstructing the 'memory' of who the client is and what state they are in.

Working Together

In practice, cookies and sessions work as a pair: the cookie acts as a portable 'key' carried by the client, while the session is the actual 'locked box' of data kept safely on the server that the key unlocks. This combination lets stateless HTTP requests be woven together into a continuous, stateful user experience — such as staying logged in across multiple pages of a website.

Q1. What is a web server? How does HTTPS ensure secure communication between clients and servers? Provide a brief explanation of the HTTP request and response mechanism. [8 Marks]

Definition

A web server is a software application (running on hardware) that stores, processes, and delivers web pages/resources to clients (browsers) upon request, typically communicating using the HTTP or HTTPS protocol.

How HTTPS Ensures Secure Communication

HTTPS (HTTP Secure) wraps standard HTTP communication inside an SSL/TLS encryption layer. During the initial handshake, the server presents a digital certificate proving its identity, and both client and server negotiate a shared symmetric encryption key. All subsequent data is encrypted using this key, protecting it from eavesdropping (confidentiality), tampering (integrity), and verifying the server's identity (authentication).

HTTP Request-Response Cycle

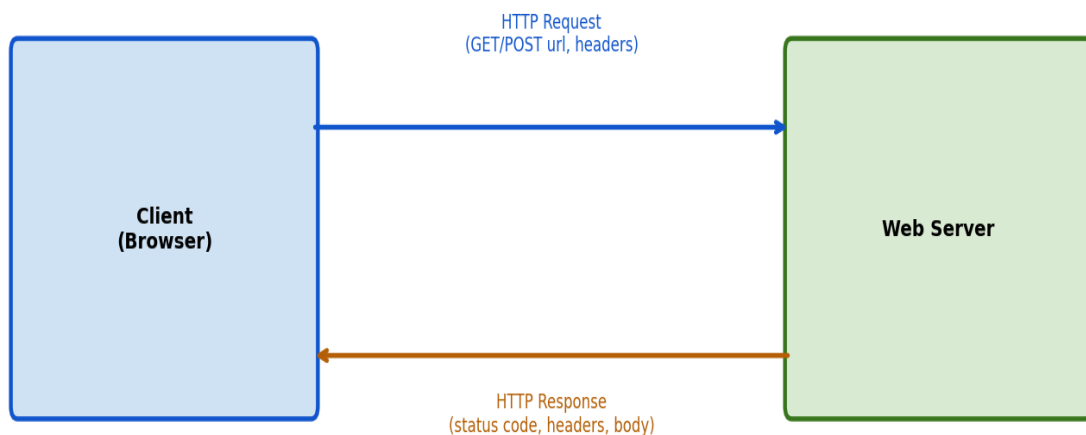


Figure: HTTP Request-Response Cycle

HTTP Request and Response Mechanism

A client sends an HTTP request to the server, consisting of a method (GET, POST, etc.), a URL, headers, and optionally a body. The server processes this request and returns an HTTP response, consisting of a status code (e.g., 200 OK, 404 Not Found), response headers, and the requested content (body) — completing one request-response cycle.

Q2. What is the MVC (Model-View-Controller) design pattern? How does it contribute to the development of web applications? Provide an explanation of two frameworks that follow the MVC pattern. [8 Marks]

MVC (Model-View-Controller) Architecture

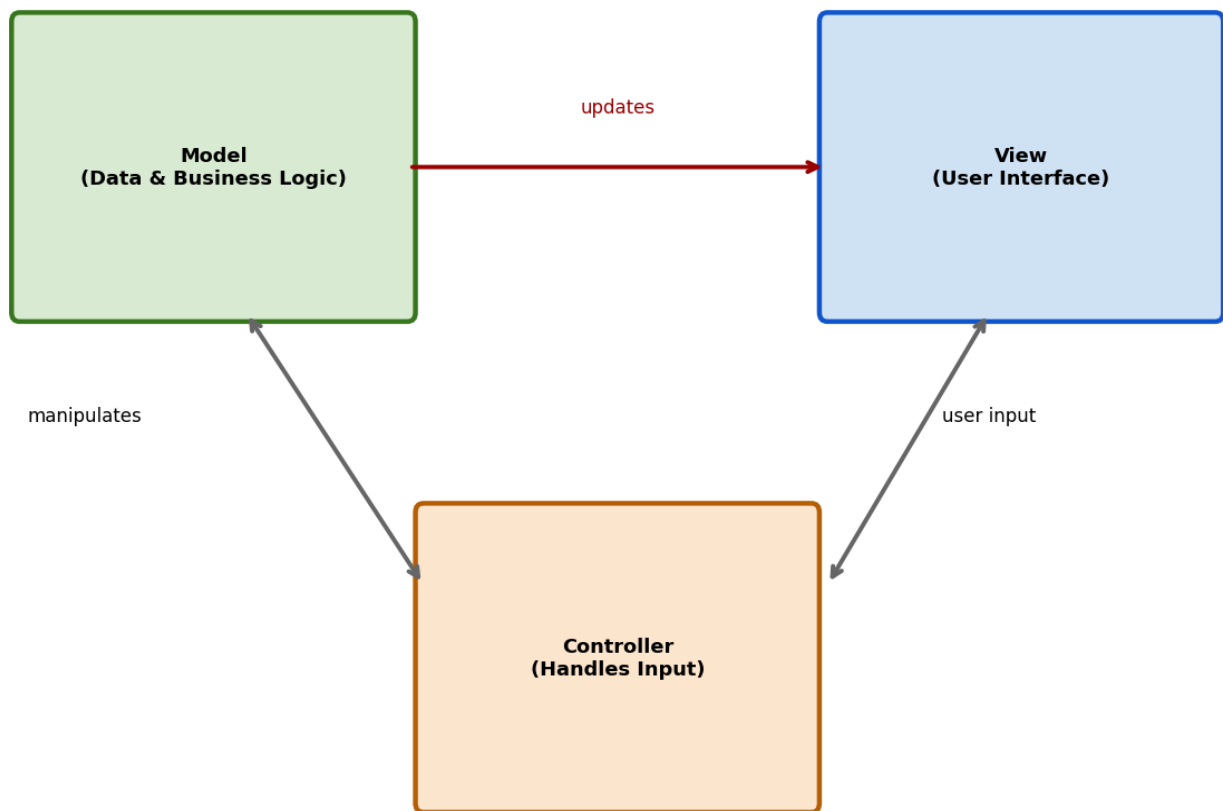


Figure: MVC Architecture

MVC Design Pattern

MVC separates an application into three components: the Model (data and business logic), the View (presentation/UI), and the Controller (handles input and mediates between Model and View).

Contribution to Web Application Development

- Separation of concerns makes code more organized, testable, and maintainable
- Multiple developers can work on the Model, View, and Controller simultaneously with minimal conflict
- Changes to the UI (View) don't require changes to business logic (Model), and vice versa
- Encourages reusability of business logic across different views/interfaces

Two Frameworks Following MVC

- **Django (Python):** Follows an MVC-inspired pattern often called MVT (Model-View-Template), where Django's 'View' acts as the Controller and 'Template' acts as the View.
- **Ruby on Rails (Ruby):** A classic, strict implementation of MVC, widely credited with popularizing the pattern for web frameworks through 'convention over configuration'.

Q3. What are the uses of CMS? Explain popular CMS used in the market today for digital content management. [8 Marks]

Uses of CMS

- Enables non-technical users to create, edit, and publish web content without coding
- Centralizes management of large volumes of content (blogs, product pages, media)
- Supports collaborative workflows with multiple authors/editors and role-based permissions
- Speeds up website development through pre-built themes, templates, and plugins

Popular CMS in the Market

WordPress (most widely used, PHP-based, blogs and business sites), Joomla (medium complexity, community/social platforms), Drupal (highly customizable, large enterprise sites), Magento (e-commerce focused), and Shopify (fully-hosted e-commerce SaaS platform) — see the detailed comparison table in the Year 2026 section.

Q4. Define Web Service. Create a simple RESTful web service endpoint for Create and Read all records of the student's object having (id, name, email, and gpa) as attributes. [8 Marks]

Definition

A web service is a standardized way for two applications to communicate and exchange data over a network (typically the internet), using well-defined protocols (SOAP or REST) and formats (XML or JSON), regardless of the underlying platform or programming language.

Code Example — Flask RESTful Endpoint

```
from flask import Flask, request, jsonify

app = Flask(__name__)
students = [] # in-memory store: [{id, name, email, gpa}, ...]
next_id = 1

@app.route('/students', methods=['POST']) # Create
def create_student():
    global next_id
    data = request.get_json()
    student = {
        'id': next_id,
        'name': data['name'],
        'email': data['email'],
        'gpa': data['gpa']
    }
    students.append(student)
    next_id += 1
    return jsonify(student), 201

@app.route('/students', methods=['GET']) # Read all
def get_students():
    return jsonify(students), 200

if __name__ == '__main__':
    app.run(debug=True)
```

Q5. What do you mean by Vulnerabilities, Threats, Attacks in web application? How do you protect from them? [8 Marks]

Definitions

Vulnerability: A weakness in a system that could be exploited to compromise its security (e.g., outdated software, unvalidated input).

Threat: A potential source of harm that could exploit a vulnerability (e.g., a malicious actor, malware).

Attack: The actual, deliberate action taken to exploit a vulnerability (e.g., an SQL injection attempt, a phishing email).

Protection Measures

- Input validation and sanitization on both client and server side
- Use parameterized queries/ORMs to prevent injection attacks
- Enforce HTTPS to protect data in transit
- Implement strong authentication (multi-factor authentication) and authorization (RBAC)
- Keep frameworks, libraries, and dependencies updated with security patches
- Conduct regular security audits and penetration testing

Q6. What do you mean by Web Application Security? Explain about the best security practices for web development. [8 Marks]

Definition

Web Application Security refers to the set of measures, practices, and technologies employed to protect web applications from vulnerabilities, threats, and attacks that could compromise their confidentiality, integrity, or availability.

Best Security Practices

- **Input Validation:** Never trust user input; validate and sanitize on the server side even if client-side validation exists.
- **Use HTTPS Everywhere:** Encrypt all data in transit using SSL/TLS.
- **Secure Password Storage:** Store password hashes (with salting), never plain text.
- **Principle of Least Privilege:** Grant users/services only the minimum permissions necessary.
- **Keep Software Updated:** Regularly patch frameworks, libraries, and server software.
- **Use Security Headers:** Implement headers like Content-Security-Policy and X-Frame-Options to mitigate XSS/clickjacking.
- **Session Management:** Use secure, HttpOnly, time-limited session cookies.
- **Regular Security Testing:** Perform vulnerability scanning and penetration testing periodically.

Year 2025 — Pre-University Examination, NAST (Sem IV Spring)

Q1. Define client-server programming. Also, differences between client-side and server-side technologies with code example. [8 Marks]

Definition

Client-server programming is a programming approach where an application is split into two cooperating parts — a client that requests services/resources, and a server that provides them — communicating over a network.

Client-Side vs Server-Side Technologies

Aspect	Client-Side	Server-Side
Executes On	The user's browser	The web server
Purpose	Handles UI, interactivity, immediate feedback	Handles business logic, database access, security
Technologies	HTML, CSS, JavaScript	Python (Flask/Django), PHP, Node.js, Java
Visibility	Source code visible to the user (view-source)	Source code hidden from the user

Code Example — Client-Side (JavaScript)

```
<script>
  function greet() {
    alert("Hello from the client-side!");
  }
</script>
<button onclick="greet()">Click Me</button>
```

Code Example — Server-Side (Python/Flask)

```
@app.route('/greet')
def greet():
    return "Hello from the server-side!"    # generated on the server, sent to client
```

Q2. What do you mean by Python file handling? Write down the code to write and read back a file in a console. [8 Marks]

In Simple English

File handling means using code to create, write to, read from, or modify files stored on disk, instead of just working with data temporarily in memory.

Definition

File handling in Python refers to the set of built-in operations — opening, reading, writing, and closing files — that allow a program to persist data to disk or retrieve previously stored data, typically using the built-in `open()` function.

Code Example

```
# Writing to a file
with open("notes.txt", "w") as f:
    f.write("Hello, this is my first line.\n")
    f.write("Python file handling is easy.\n")

# Reading back the file and printing to console
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)
```

Using 'with open(...)' automatically closes the file once the block finishes, even if an error occurs, avoiding resource leaks.

Q3. Define class and object in Python. Explain how class, object, and inheritance are created in Python with a code example. [8 Marks]

Definitions

Class: A blueprint/template that defines the attributes (data) and methods (behavior) that its objects will have.

Object: A specific instance of a class, created in memory with its own actual attribute values.

Code Example

```
class Animal:                                # Base class
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound.")

class Dog(Animal):                            # Derived class (inheritance)
    def speak(self):
        print(f"{self.name} says Woof!")

a = Animal("Generic Animal")                 # Creating an object of Animal
d = Dog("Tommy")                             # Creating an object of Dog
a.speak()                                    # Generic Animal makes a sound.
d.speak()                                    # Tommy says Woof!
```

Here, Dog inherits from Animal (acquiring its `__init__` constructor), but overrides the `speak()` method to provide its own specific behavior — demonstrating both inheritance and method overriding (polymorphism).

Q4. Define Flask. Write down how you create a form and handle request and response in Flask. [8 Marks]

Definition

Flask is a lightweight, micro web framework for Python that provides the essential tools (routing, request handling, templating) needed to build web applications, while remaining minimal and highly extensible.

Code Example — Form Handling

```
from flask import Flask, render_template, request
app = Flask(__name__)

@app.route('/contact', methods=['GET', 'POST'])
def contact():
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        return f"Thanks {name}, we will reach you at {email}!"
    return render_template('contact.html')
```

contact.html

```
<form method="POST">
  <input type="text" name="name" placeholder="Your Name">
  <input type="email" name="email" placeholder="Your Email">
  <button type="submit">Submit</button>
</form>
```

On a GET request, Flask renders the empty form. On a POST request (form submission), request.form is used to read the submitted field values, and a response is generated and returned to the client.

Q5. What is the need of form validation in web applications? Write down the code showing how many ways you can validate user input. [8 Marks]

In Simple English

Form validation checks that whatever a user typed into a form actually makes sense (a valid email, a non-empty name) before the application processes or saves it — protecting both the user experience and the system's data integrity/security.

Need for Form Validation

- Prevents invalid, incomplete, or malicious data from being saved to the database
- Improves user experience by catching mistakes immediately with clear feedback
- Forms a critical layer of defense against attacks like SQL Injection and XSS
- Ensures data consistency and correctness for downstream processing

Ways to Validate User Input

1. Client-Side Validation (HTML5 attributes):

```
<input type="email" required minlength="5" maxlength="50">
```

2. Client-Side Validation (JavaScript):

```
if (document.getElementById('name').value === '') {  
    alert('Name cannot be empty!');  
}
```

3. Server-Side Validation (Python/Flask):

```
@app.route('/register', methods=['POST'])  
def register():  
    email = request.form['email']  
    if '@' not in email:  
        return "Invalid email address!", 400  
    return "Registration successful!"
```

4. Server-Side Validation (using Flask-WTF form validators):

```
from flask_wtf import FlaskForm  
from wtforms import StringField  
from wtforms.validators import DataRequired, Email, Length  
  
class RegisterForm(FlaskForm):  
    name = StringField('Name', validators=[DataRequired(), Length(min=2, max=50)])  
    email = StringField('Email', validators=[DataRequired(), Email()])
```

Server-side validation should always be performed, even when client-side validation exists, since client-side checks can be bypassed by a malicious user.

Q6. Write about message flashing and file uploads with code example. *[8 Marks]*

Message Flashing

Message flashing is a Flask feature that lets you record a message at the end of one request and display it exactly once at the beginning of the next request — commonly used to show a 'success' or 'error' message after form submission (e.g., 'Login successful!').

```
from flask import flash, redirect, url_for, render_template

@app.route('/login', methods=['POST'])
def login():
    flash('Login successful!')
    return redirect(url_for('dashboard'))

# In the template:
# {% with messages = get_flashed_messages() %}
#   {% for message in messages %}
#     <p>{{ message }}</p>
#   {% endfor %}
# {% endwith %}
```

File Uploads

Flask handles file uploads through the `request.files` object, allowing users to submit files (e.g., images, documents) via a multipart form.

```
from werkzeug.utils import secure_filename
import os

@app.route('/upload', methods=['POST'])
def upload():
    file = request.files['profile_pic']
    filename = secure_filename(file.filename)
    file.save(os.path.join('uploads', filename))
    return 'File uploaded successfully!'
```

The corresponding HTML form must include `enctype="multipart/form-data"` for file uploads to work: `<form method="POST" enctype="multipart/form-data">`.

Q7. Design a signup page having 4-5 fields and encrypt the password, saving all the information to a database using SQLAlchemy. [8 Marks]

(This code is identical to the full solution given in the Year 2026 section, Q6 — repeated here for reference.)

```
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    full_name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False)
    phone = db.Column(db.String(20))
    address = db.Column(db.String(200))
    password_hash = db.Column(db.String(200), nullable=False)

@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        new_user = User(
            full_name=request.form['full_name'],
            email=request.form['email'],
            phone=request.form['phone'],
            address=request.form['address'],
            password_hash=generate_password_hash(request.form['password'])
        )
        db.session.add(new_user)
        db.session.commit()
        return redirect(url_for('signup'))
    return render_template('signup.html')
```

Q9. Define SOA. Explain SOAP components with code example. [8 Marks]

In Simple English

SOA is a way of designing software as a collection of independent, reusable services that talk to each other over a network. SOAP is one common protocol used by such services to actually exchange messages, following a strict, structured XML format.

SOA (Service-Oriented Architecture)

Service-Oriented Architecture is a software design approach where application functionality is organized as a collection of independent, loosely-coupled, reusable services, each performing a specific business function and communicating with one another over a network using standardized protocols.

SOAP Components

- **Envelope:** The root element that identifies the XML document as a SOAP message.
- **Header (optional):** Contains metadata about the message, such as authentication tokens or transaction information.
- **Body:** Contains the actual request or response data — the core message content.
- **Fault (optional):** Contains error/status information if something went wrong processing the message.

Code Example — SOAP Message

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header>
    <AuthToken>abc123</AuthToken>
  </soap:Header>
  <soap:Body>
    <GetStudentRequest>
      <StudentID>101</StudentID>
    </GetStudentRequest>
  </soap:Body>
</soap:Envelope>
```

Q10. Define web vulnerabilities. Explain design guidelines for secure web applications. [8 Marks]

Definition

A web vulnerability is a weakness or flaw in a web application's design, implementation, or configuration that could be exploited by an attacker to compromise its security — for example, unvalidated input, weak authentication, or exposed sensitive data.

Design Guidelines for Secure Web Applications

- **Validate All Input:** Treat all user input as untrusted; validate and sanitize on the server side.
- **Use Secure Authentication:** Enforce strong password policies, hashing (never plain-text storage), and consider multi-factor authentication.
- **Apply Least Privilege:** Grant users and processes only the minimum access rights necessary.
- **Encrypt Sensitive Data:** Use HTTPS for data in transit, and encrypt sensitive data at rest.
- **Secure Session Management:** Use secure, time-limited, HttpOnly cookies for session IDs.
- **Error Handling:** Avoid exposing detailed internal error messages/stack traces to end users.
- **Keep Dependencies Updated:** Regularly patch frameworks and third-party libraries against known vulnerabilities.

Q12. What is Gunicorn? Briefly write about the uses of monitoring and logging. [8 Marks]

Gunicorn

(See Year 2026 section, Q10, for the full definition and example command.)

Production Deployment: Nginx + Gunicorn + Flask

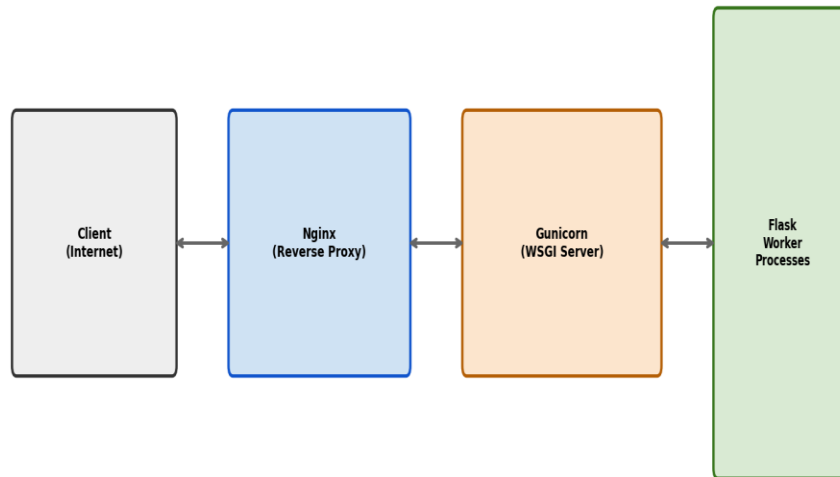


Figure: Production Deployment — Nginx + Gunicorn + Flask

Uses of Monitoring and Logging

- **Monitoring:** Continuously tracks the health and performance of a running application (CPU/memory usage, response times, error rates), enabling teams to detect and respond to issues (e.g., server overload, crashes) proactively, often via dashboards and automated alerts (e.g., using tools like Prometheus, Grafana, or New Relic).
- **Logging:** Records events, errors, and activity occurring within the application over time, providing a detailed history useful for debugging issues after they occur, auditing user actions, and understanding usage patterns.

Together, monitoring and logging give development/operations teams the visibility needed to maintain application reliability, quickly diagnose production issues, and continuously improve performance.

Q13. Write short notes on: DoS and SQLI. [8 Marks]

DoS (Denial of Service): An attack that attempts to make a system or service unavailable to legitimate users by overwhelming it with excessive traffic or requests, exhausting its resources (bandwidth, memory, CPU). A Distributed DoS (DDoS) uses many compromised machines simultaneously to amplify the attack.

SQLI (SQL Injection): An attack where malicious SQL code is inserted into an input field and executed by the backend database, potentially allowing unauthorized data access, modification, or deletion. Prevented primarily through parameterized queries/ORMs and input validation.

Q14. Write short notes on: Deployment platforms. [8 Marks]

In Simple English

Deployment platforms are the services/environments where a finished web application is actually hosted and made available to real users on the internet.

Technical Explanation

Deployment platforms are hosting environments and services used to publish and run web applications so they are accessible to end users. Options range from traditional VPS/dedicated servers to modern cloud and Platform-as-a-Service (PaaS) offerings.

- **PaaS (Platform as a Service):** Heroku, Render, PythonAnywhere — abstract away server management, letting developers deploy with minimal configuration.
- **Cloud Infrastructure (IaaS):** AWS EC2, Google Cloud, Azure — provide more control but require more configuration and management.
- **Containerized Deployment:** Docker + Kubernetes — packages the application with its dependencies for consistent deployment across environments.
- **Serverless Platforms:** AWS Lambda, Google Cloud Functions — run code in response to events without managing servers at all.

Q15. Write short notes on: Differentiate between function and method in Python. [8 Marks]

Aspect	Function	Method
Definition	A block of reusable code defined independently using 'def'	A function defined inside a class, associated with its objects
Calling	Called directly by name: my_function()	Called on an object: my_object.my_method()
First Parameter	No implicit first parameter	Implicitly receives 'self' (the instance) as its first parameter
Example	def add(a, b): return a + b	def deposit(self, amount): self.balance += amount

Q1. Define Web Development. Write about the client-side and server-side scripting used to develop a website/application with code example. [8 Marks]

Definition

Web development is the process of building, programming, and maintaining websites and web applications, encompassing everything from designing the user-facing interface to building the underlying server logic and database that powers it.

Client-Side Scripting

Client-side scripting refers to code that runs directly in the user's web browser, handling presentation, interactivity, and immediate feedback without requiring a round-trip to the server. Common technology: JavaScript.

```
<script>
  function validateForm() {
    if (document.getElementById('email').value === '') {
      alert('Please enter your email!');
      return false;
    }
  }
</script>
```

Server-Side Scripting

Server-side scripting refers to code that runs on the web server, processing requests, interacting with databases, and generating dynamic content before sending the final response to the client. Common technologies: Python (Flask/Django), PHP, Node.js.

```
@app.route('/user/<int:id>')
def get_user(id):
    user = User.query.get(id)      # queries the database on the server
    return render_template('profile.html', user=user)
```

Q2. Define UDDI. Explain about Web Service and RESTful web services in detail. [8 Marks]**Definition of UDDI**

UDDI (Universal Description, Discovery, and Integration) is an XML-based registry standard that allows businesses to publish and discover information about available web services over the internet — acting like a searchable 'directory' of web services, typically used alongside WSDL (which describes the service) and SOAP (which invokes it).

Web Service

A web service is a standardized way for applications to communicate and exchange data over a network, using well-defined protocols (SOAP or REST) and data formats (XML/JSON), independent of platform or programming language.

RESTful Web Services

RESTful web services follow the REST (Representational State Transfer) architectural style, using standard HTTP methods (GET, POST, PUT, DELETE) to operate on resources identified by URLs, typically exchanging data in lightweight JSON format. They are stateless — each request contains all information needed to process it, with no session state stored on the server between requests.

- Resources are identified by URLs (e.g., /students/5)
- Standard HTTP methods represent CRUD operations (GET=read, POST=create, PUT=update, DELETE=delete)
- Stateless: each request is independent and self-contained
- Lightweight and widely supported, making REST the dominant style for modern web APIs (compared to the heavier SOAP protocol)

Q3. Explain Network threats and host threats. Also, write down some countermeasures for both of them. [8 Marks]

Network Threats

Network threats target the communication infrastructure and data in transit between systems, aiming to intercept, disrupt, or manipulate network traffic.

- **Eavesdropping/Sniffing:** Intercepting data as it travels across the network.
- **Man-in-the-Middle (MITM) Attacks:** Secretly intercepting and possibly altering communication between two parties.
- **Denial of Service (DoS/DDoS):** Overwhelming network resources to make services unavailable.
- **IP Spoofing:** Forging the source IP address of packets to impersonate another system.

Host Threats

Host threats target the individual server/machine itself, aiming to compromise the system directly.

- **Malware/Viruses:** Malicious software that damages or exploits a host system.
- **Unauthorized Access:** Attackers gaining access through weak/stolen credentials.
- **Privilege Escalation:** Exploiting a vulnerability to gain higher-level access than intended.
- **Buffer Overflow Attacks:** Exploiting memory-handling flaws to execute malicious code.

Countermeasures

Threat Type	Countermeasures
Network Threats	Use HTTPS/TLS encryption, firewalls, intrusion detection systems (IDS), VPNs, and network segmentation
Host Threats	Keep systems patched/updated, use antivirus/anti-malware, enforce strong access controls, apply the principle of least privilege, conduct regular security audits

Q4. Define Authentication and Authorization. Explain different threats and countermeasures you need to take care about for any web application. [8 Marks]

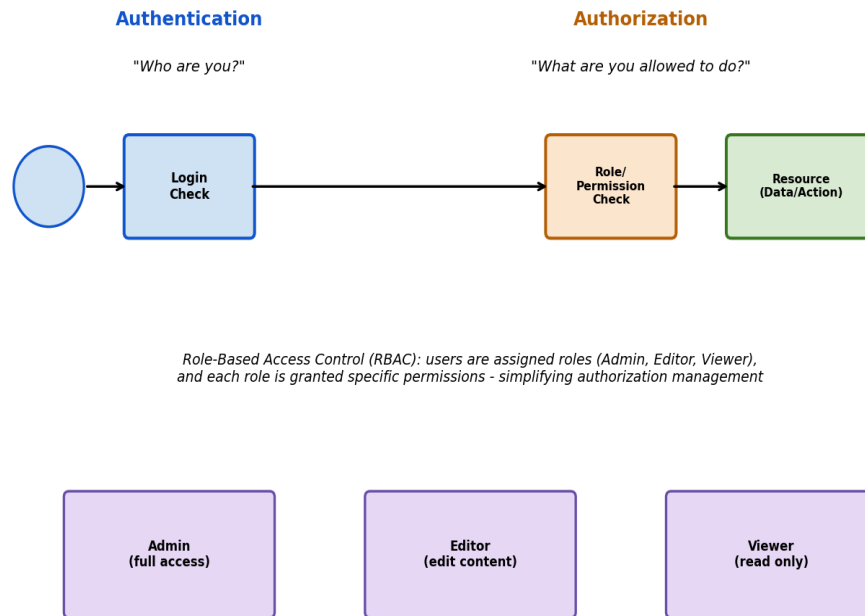


Figure: Authentication vs Authorization

Definitions

Authentication: The process of verifying a user's identity (e.g., through a username/password) — 'Who are you?'

Authorization: The process of determining what an authenticated user is permitted to do — 'What are you allowed to do?'

Common Web Application Threats and Countermeasures

Threat	Countermeasure
SQL Injection	Use parameterized queries/ORMs; validate all input
Cross-Site Scripting (XSS)	Escape/sanitize user-generated output; use Content-Security-Policy headers
Cross-Site Request Forgery (CSRF)	Use anti-CSRF tokens on all state-changing forms
Broken Authentication	Enforce strong passwords, hashing, multi-factor authentication, secure session management
Sensitive Data Exposure	Encrypt data in transit (HTTPS) and at rest; avoid storing unnecessary sensitive data
Broken Access Control	Implement proper Role-Based Access Control (RBAC) and verify authorization on every request

Q5. Write short notes on: Java Framework. [8 Marks]

In Simple English

A Java framework is a pre-built toolkit that gives Java developers a ready-made structure and set of reusable components for building web applications, instead of writing every piece of infrastructure from scratch.

Technical Explanation

Java frameworks provide reusable, standardized components for building enterprise-grade web applications in Java, handling common concerns such as dependency injection, MVC routing, ORM/data access, and security.

- **Spring / Spring Boot:** The most widely used Java framework, providing dependency injection, MVC support, and rapid application setup (Spring Boot); dominant in enterprise Java development.
- **Hibernate:** An ORM framework for Java, simplifying database interaction by mapping Java objects to relational database tables.
- **JavaServer Faces (JSF):** A component-based UI framework for building Java web applications.
- **Struts:** An older, MVC-based framework for building Java web applications.

Q6. Write short notes on: Foundation of security. [8 Marks]

In Simple English

The foundation of security refers to the core, fundamental principles that all good security practices are built on — protecting data from being seen, changed, or made unavailable by the wrong people.

Technical Explanation — The CIA Triad

The foundation of information/web security rests on three core principles, commonly known as the CIA Triad:

- **Confidentiality:** Ensuring information is accessible only to those authorized to access it (e.g., through encryption and access controls).
- **Integrity:** Ensuring information is accurate and has not been tampered with or corrupted (e.g., through checksums, digital signatures).
- **Availability:** Ensuring authorized users can access information and resources when needed (e.g., through redundancy, DoS protection).

Additional foundational principles include Authentication (verifying identity), Authorization (controlling access), and Non-repudiation (ensuring an action cannot be denied after the fact) — together forming the basis on which all specific security measures and practices are designed.

Q1. What are the basic protocols used in the internet? Differentiate between Client-Side and Server-Side Scripting. [8 Marks]

Basic Internet Protocols

Protocol	Purpose
HTTP/HTTPS	Transfers web pages/resources between client and server (HTTPS adds encryption)
TCP/IP	Core protocol suite for reliable data transmission across networks
FTP	Transfers files between a client and server
SMTP/POP3/IMAP	Sending and receiving email
DNS	Translates human-readable domain names into IP addresses
SSL/TLS	Provides encryption and secure communication over a network

Client-Side vs Server-Side Scripting

Aspect	Client-Side Scripting	Server-Side Scripting
Runs On	The browser	The web server
Purpose	UI interactivity, immediate validation/feedback	Business logic, database operations, security
Technologies	JavaScript, HTML, CSS	PHP, Python, Node.js, Java
Source Visibility	Visible to the end user	Hidden from the end user

Q2. Write a server-side script that can insert and retrieve data from the 'student' table of a MySQL database server. [8 Marks]

Code Example (FLASK with MySQLi)

SQL

```
CREATE DATABASE college;

USE college;

CREATE TABLE student (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    address VARCHAR(100),
    age INT
);
```

App.py

```
from flask import Flask, request, render_template
import mysql.connector

app = Flask(__name__)

# MySQL connection
db = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="college"
)

# Retrieve data
@app.route("/")
def home():
    cursor = db.cursor()
    cursor.execute("SELECT * FROM student")
    students = cursor.fetchall()
    cursor.close()

    return render_template("index.html", students=students)

# Insert data
@app.route("/insert", methods=["POST"])
def insert():
    name = request.form["name"]
    address = request.form["address"]
    age = request.form["age"]

    cursor = db.cursor()

    query = "INSERT INTO student (name, address, age) VALUES (%s, %s, %s)"
    values = (name, address, age)

    cursor.execute(query, values)
    db.commit()
    cursor.close()
```

```
return "Student data inserted successfully!"
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

templates/index.html

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Student</title>  
</head>  
<body>  
  
<h2>Insert Student</h2>  
  
<form action="/insert" method="POST">  
    Name: <input type="text" name="name"><br><br>  
    Address: <input type="text" name="address"><br><br>  
    Age: <input type="number" name="age"><br><br>  
  
    <input type="submit" value="Insert">  
</form>  
  
<h2>Student Records</h2>  
  
<table border="1">  
    <tr>  
        <th>ID</th>  
        <th>Name</th>  
        <th>Address</th>  
        <th>Age</th>  
    </tr>  
  
    {% for student in students %}  
    <tr>  
        <td>{{ student[0] }}</td>  
        <td>{{ student[1] }}</td>  
        <td>{{ student[2] }}</td>  
        <td>{{ student[3] }}</td>  
    </tr>  
    {% endfor %}  
</table>  
  
</body>  
</html>
```

A prepared statement (bind_param) is used for the INSERT operation rather than directly concatenating variables into the SQL string, protecting the script against SQL Injection attacks.

Q3. What is a Content Management System? How does CMS help manage content? Also, list out some advantages of CMS. [8 Marks]

Definition

A Content Management System (CMS) is a software application that enables users to create, manage, edit, and publish digital content on a website through a user-friendly interface, without requiring specialized coding knowledge.

How CMS Helps Manage Content

- Provides a WYSIWYG editor for creating and formatting content visually
- Organizes content using categories, tags, and structured content types
- Manages media assets (images, videos, documents) in a central library
- Controls user roles and permissions for multi-author collaboration
- Handles version history, allowing content to be reviewed or reverted

Advantages of CMS

- Enables non-technical users to publish and update content independently
- Speeds up website development using pre-built themes and plugins
- Provides built-in SEO tools to improve content visibility in search engines
- Supports collaborative multi-user workflows with defined roles
- Large community support and extensive documentation for popular platforms

Q4. What is WSDL? Explain its uses in web technology and write its structure. [8 Marks]

Definition

WSDL (Web Services Description Language) is an XML-based language used to describe the functionality, location, and technical details of a web service — essentially a machine-readable 'contract' specifying what operations a web service offers and how to invoke them.

Uses in Web Technology

- Allows client applications to automatically discover and understand what operations a SOAP web service provides
- Specifies the exact data types, message formats, and protocols required to communicate with the service
- Enables automated generation of client-side proxy code from the WSDL document
- Works alongside UDDI (for discovery) and SOAP (for message exchange) to enable interoperable web services

Structure of a WSDL Document

- **<types>**: Defines the data types used in the messages (usually via XML Schema).
- **<message>**: Defines the data elements of each operation (input/output).
- **<portType>**: Defines the set of operations supported, and the messages involved in each.
- **<binding>**: Specifies the protocol and data format for each operation (e.g., SOAP over HTTP).
- **<service>**: Specifies the address (URL) where the service can be accessed.

Q5. List web application security fundamentals. Explain host threats and application threats in detail. [8 Marks]

Web Application Security Fundamentals

- Input validation and sanitization
- Secure authentication and session management
- Proper authorization/access control (RBAC)
- Encryption of data in transit (HTTPS) and at rest
- Regular security testing and patching
- Secure error handling and logging

Host Threats

Host threats target the server/machine hosting the application directly — including malware infections, unauthorized access via weak credentials, privilege escalation exploits, and buffer overflow attacks that compromise the underlying operating system or server software.

Application Threats

Application threats target vulnerabilities within the web application's own code and logic — including SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), broken authentication, and insecure direct object references (IDOR), which exploit flaws in how the application processes and handles data.

Q7. What is MVC? What are the advantages of using the framework? Explain two PHP frameworks. [8 Marks]

MVC (Model-View-Controller) Architecture

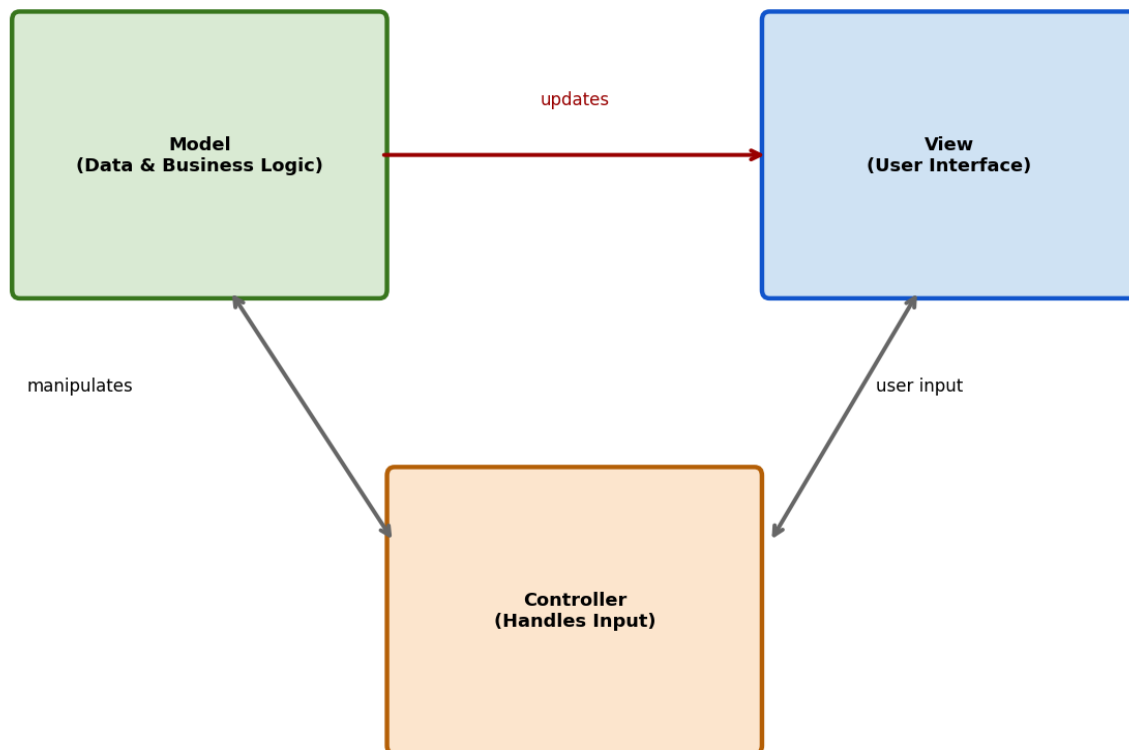


Figure: MVC Architecture

MVC

MVC (Model-View-Controller) is a software design pattern that separates an application into three components: the Model (data/business logic), View (user interface), and Controller (handles input and coordinates between Model and View).

Advantages of Using an MVC Framework

- Clear separation of concerns, making the codebase easier to understand and maintain
- Enables parallel development — different developers can work on the Model, View, and Controller simultaneously
- Improves testability, since business logic (Model) can be tested independently of the UI
- Promotes code reuse across different views/interfaces sharing the same underlying Model

Two PHP Frameworks Following MVC

- **Laravel:** A modern, widely-used PHP framework known for its elegant syntax, built-in ORM (Eloquent), templating engine (Blade), and comprehensive tools for routing, authentication, and testing — making it popular for rapid, secure web application development.
- **CodeIgniter:** A lightweight, fast PHP framework with a small footprint and minimal configuration requirements, well-suited for building simple to medium-complexity applications quickly, with straightforward MVC structure.

Q8. Write short notes on: GET and POST. [8 Marks]**In Simple English**

GET and POST are the two most common ways a browser sends a request to a server — GET is for asking for/retrieving data, and POST is for sending data to be processed or saved.

Aspect	GET	POST
Purpose	Retrieve data from the server	Send data to the server to create/update a resource
Data Visibility	Appended to the URL (visible, bookmarkable)	Sent in the request body (not visible in the URL)
Data Size Limit	Limited by URL length (~2000 characters)	Much larger limit, suitable for file uploads and forms
Idempotency/Safety	Safe and idempotent — should not change server state	Not idempotent — typically changes server state
Caching	Can be cached by browsers	Not typically cached

Q9. Write short notes on: SOAP. [8 Marks]

SOAP vs REST		
	SOAP	REST
Protocol	Strict protocol (XML-based)	Architectural style (any format, usually JSON)
Transport	Can use HTTP, SMTP, others	Typically HTTP only
Message Format	XML (Envelope, Header, Body)	JSON, XML, plain text
State	Can be stateful or stateless	Stateless by design
Performance	Heavier, more verbose	Lighter, faster

Figure: SOAP vs REST

SOAP (Simple Object Access Protocol) is a strict, XML-based messaging protocol for exchanging structured information between applications over a network. Each SOAP message consists of an Envelope (the root element), an optional Header (metadata), a Body (the actual request/response content), and an optional Fault (error information). SOAP can operate over multiple transport protocols (HTTP, SMTP) and supports built-in error handling and strong typing via XML Schema, making it well-suited for enterprise applications requiring strict contracts, formal security standards (WS-Security), and transactional reliability — though it is more verbose and complex than the lighter-weight REST style.